

Queens Puzzle Synthesis and Verification Engine

Abstract

The **Queens Puzzle Synthesis and Verification Engine** is a computational system designed to automatically generate valid, unique, and structurally diverse instances of grid-based Queens logic puzzles. The project addresses the fundamental difficulty of procedural puzzle generation: producing boards that are simultaneously **solvable, uniquely solvable, spatially contiguous, and sufficiently different from previously generated boards**.

A Queens puzzle consists of an $N \times N$ grid divided into N contiguous regions. The objective is to place exactly N queens such that each row, column, and region contains exactly one queen, while no two queens are adjacent horizontally, vertically, or diagonally.

Conventional procedural generation approaches generally follow a **region-first strategy**, in which the grid is initially divided into randomly generated regions and subsequently tested for solvability. This approach can produce boards with no valid solution, multiple solutions, or structurally redundant boards that are merely rotations or reflections of previously generated puzzles.

The proposed system reverses this process through an **inverted constraint-guided synthesis architecture**. Instead of generating arbitrary regions and attempting to find a solution afterward, the system first generates a valid non-attacking N -Queens configuration. These queen positions become fixed seed points for constructing the puzzle regions. A multi-source **Breadth-First Search (BFS) wavefront expansion** then grows the regions around these seeds while maintaining four-directional spatial connectivity.

Each generated candidate is subsequently evaluated using an exact **backtracking Constraint Satisfaction Problem (CSP) solver with constraint propagation**. The solver checks row, column, region, and Chebyshev adjacency constraints and terminates early whenever a second solution is discovered. A candidate is accepted only when exactly one solution exists.

To avoid generating visually equivalent puzzles, the system performs **Dihedral Group (D_4) symmetry canonicalization**. The eight rotations and reflections of each candidate are generated, region labels are normalized, and a canonical representation is calculated. A hash-based structure is then used to reject previously encountered structural equivalents.

The complete generation engine operates through a cooperative asynchronous architecture, allowing computationally intensive search operations to execute without blocking the user

interface. Runtime telemetry provides information such as explored states, accepted puzzles, rejected multi-solution candidates, rejected duplicates, and generation status.

The implemented frontend provides the user-facing interface for interacting with the puzzle generation system, while the generated data and application state are connected to a cloud database for persistent storage. The project therefore combines **combinatorial optimization, constraint satisfaction, graph traversal, computational geometry, asynchronous programming, and cloud-backed application architecture** into a single puzzle synthesis platform.

The system establishes a foundation for future extensions including machine-learning-based search pruning, automated difficulty estimation using human solving metrics, WebAssembly-based parallelization, and generative models for producing more complex region geometries.

Keywords: Queens Puzzle, Constraint Satisfaction Problem, N-Queens, Procedural Content Generation, BFS, Backtracking, Constraint Propagation, Graph Traversal, Dihedral Symmetry, Puzzle Uniqueness, Cloud Database.

1. Introduction

Logic puzzles provide a useful environment for studying computational search, combinatorial optimization, constraint satisfaction, and algorithmic reasoning.

The Queens puzzle considered in this project consists of an $N \times N$ grid divided into N contiguous colored regions. A player must place N queens such that four conditions are simultaneously satisfied:

1. Exactly one queen must occur in every row.
2. Exactly one queen must occur in every column.
3. Exactly one queen must occur in every region.
4. No two queens may touch horizontally, vertically, or diagonally.

The fourth condition can be represented using the Chebyshev distance:

$$\max(|r_1 - r_2|, |c_1 - c_2|) > 1$$

for every pair of queens (r_1, c_1) and (r_2, c_2) .

While solving an already-generated puzzle is itself a CSP, **generating a good puzzle is a separate and more difficult problem**. A random partition of the board does not guarantee that the resulting puzzle has a solution. Even if it has a solution, it may have several solutions, which reduces its suitability as a deterministic logic puzzle.

Therefore, this project focuses primarily on the **synthesis and verification problem** rather than only on puzzle solving.

2. Problem Statement

The problem addressed by this project is:

To develop an automated system capable of generating Queens puzzles that contain contiguous regions, possess exactly one valid solution, avoid structurally duplicated boards, and execute continuously without blocking the user interface.

A conventional region-first generator can be represented as:

```
Random Regions
  ↓
Puzzle Solver
  ↓
Accept / Reject
```

This creates a significant amount of wasted computation because many randomly generated region configurations may not satisfy the required constraints.

The proposed system reverses the order:

```
Valid Queen Placement
  ↓
Region Synthesis
  ↓
Uniqueness Verification
  ↓
Symmetry Filtering
  ↓
Accepted Puzzle
```

This change is the central architectural idea of the project.

3. Existing System

Traditional procedural puzzle generation commonly begins by dividing the grid into regions.

Typical techniques include:

- Random flood-fill
- Voronoi-based partitioning
- Stochastic region assignment
- Random connected-component construction

The resulting board is then passed to a puzzle solver.

This approach creates several problems.

3.1 Solvability Problem

A randomly generated partition does not necessarily correspond to a valid Queens placement.

As grid size increases, the probability of producing a usable puzzle can decrease substantially.

3.2 Multiple Solutions

A board may satisfy the puzzle rules in several different ways.

For a deterministic puzzle, the desired condition is:

$$S = 1$$

where S is the number of valid solutions.

Boards with:

$$S > 1$$

must therefore be rejected.

3.3 Structural Duplication

A puzzle can be transformed through:

- 90° rotation
- 180° rotation
- 270° rotation
- Horizontal reflection
- Vertical reflection
- Diagonal reflection

and still represent essentially the same puzzle structure.

Without symmetry normalization, these variants unnecessarily occupy the generated puzzle catalogue.

4. Proposed System

The proposed system uses an **inverted constraint-propagation strategy**.

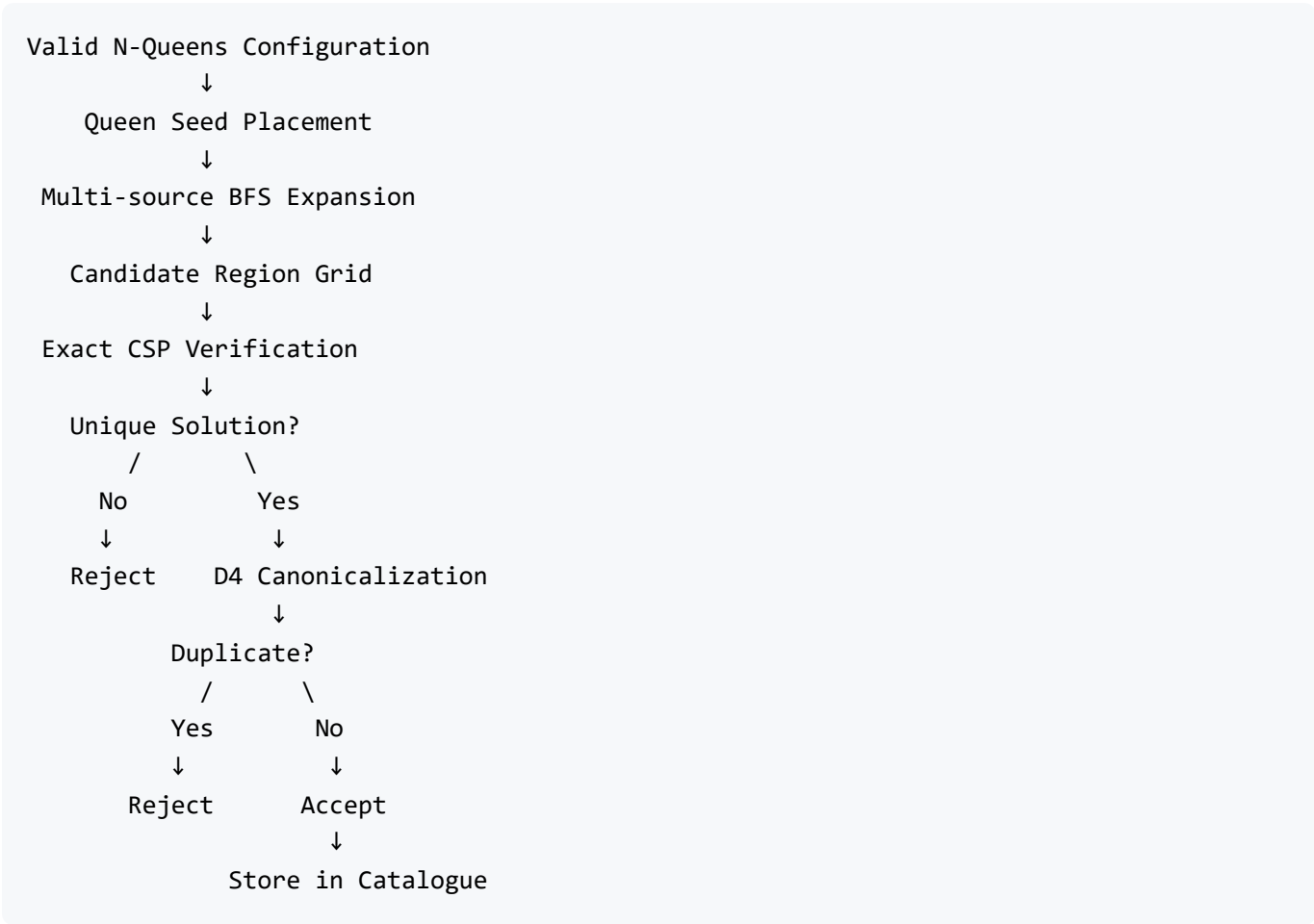
Instead of asking:

|"Can this randomly generated board be solved?"|

the system begins with:

|"Which valid queen configuration can serve as the guaranteed solution?"|

The overall pipeline is:



The synthesis process consists of four principal stages:

1. N-Queens seed generation.
2. BFS region construction.

3. CSP uniqueness verification.
 4. D_4 symmetry normalization.
-

5. Objectives

5.1 Primary Objectives

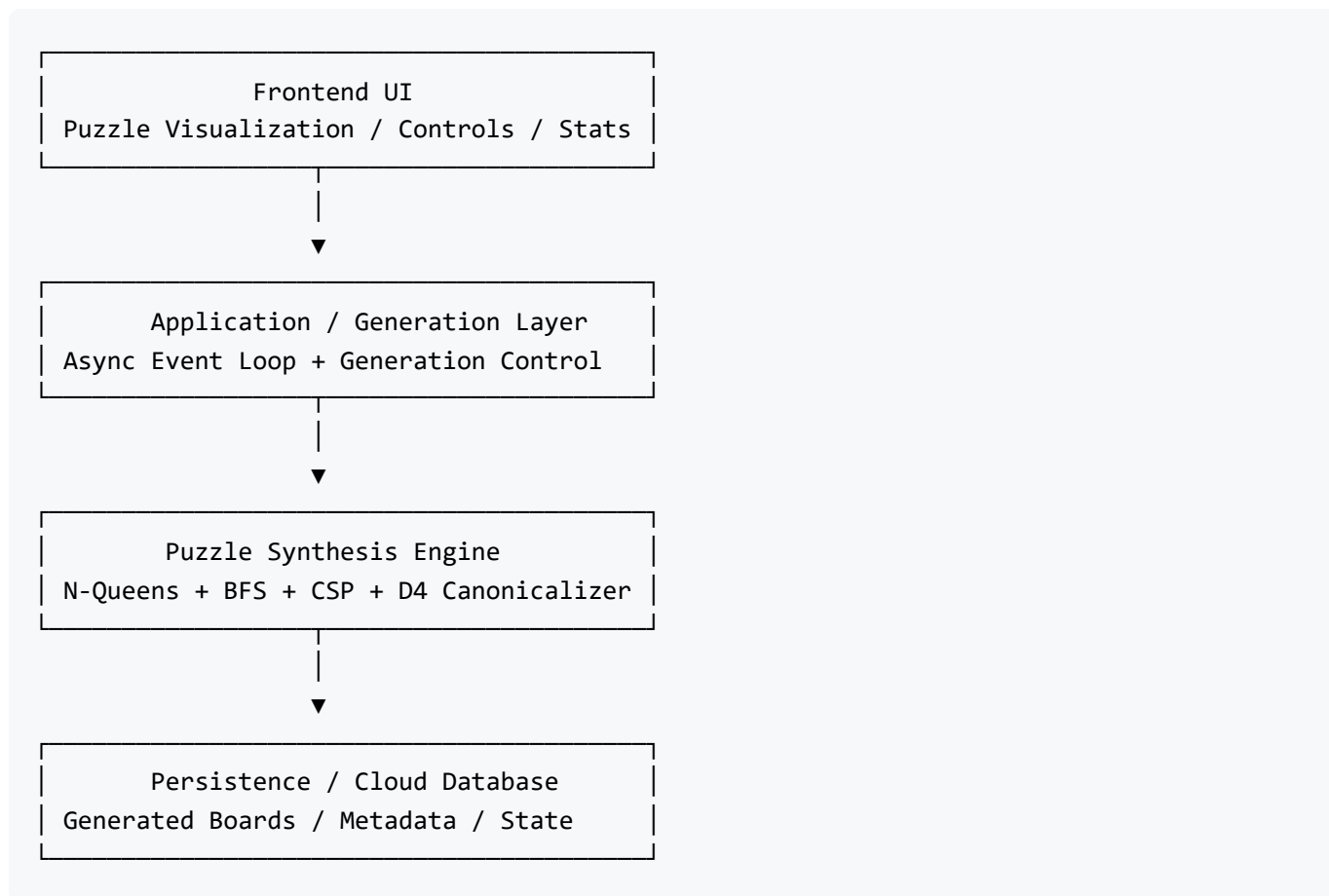
- Generate valid Queens puzzle configurations automatically.
- Guarantee that every candidate begins with at least one known valid queen placement.
- Construct spatially contiguous puzzle regions.
- Verify that every accepted puzzle has exactly one solution.
- Reject multi-solution boards.
- Detect rotational and reflective duplicates.
- Provide asynchronous generation without freezing the user interface.
- Maintain generation telemetry.
- Persist generated puzzle information through the application's cloud-connected database.

5.2 Secondary Objectives

- Provide an interactive frontend for puzzle generation and management.
 - Support long-running generation processes.
 - Allow generated puzzle data to be retained and retrieved.
 - Establish an architecture that can later support machine-learning-based optimization.
-

6. System Architecture

The system can be conceptually divided into five major layers.



The algorithmic core consists of four principal stages:

1. N-Queens seed generation.
2. BFS wavefront expansion.
3. CSP uniqueness verification.
4. Dihedral symmetry normalization.

7. Module 1 — N-Queens Seed Generator

The first stage generates a valid arrangement of queens.

The queen positions can be represented using a permutation:

$$C = [c_0, c_1, \dots, c_{N-1}]$$

where:

$$(k, c_k)$$

represents the queen position in row k .

The permutation condition ensures that two queens do not share a column:

$$c_i \neq c_j \quad \text{for } i \neq j$$

The Chebyshev constraint is:

$$\max(|i - j|, |c_i - c_j|) > 1 \quad \text{for } i \neq j$$

This guarantees that the generated queen seeds do not touch each other.

This gives the generator a significant advantage: **the initial configuration already satisfies the fundamental queen constraints.**

8. Module 2 — BFS Wavefront Region Generation

Once queen positions are known, each queen becomes a region seed.

The system then uses **multi-source BFS**.

All queen seeds are inserted into the BFS queue simultaneously:

$$Q = \{q_0, q_1, \dots, q_{N-1}\}$$

The BFS explores orthogonal neighbors:

$$\mathcal{N}_4(r, c) = \{(r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)\}$$

subject to grid boundaries: $0 \leq r < N$ and $0 \leq c < N$.

The important property is that cells are processed according to their topological relationship with already discovered cells.

Therefore, when a cell becomes a candidate for assignment, it already has an adjacent cell belonging to an existing region.

This helps preserve:

$$\text{Region Connectivity} = \text{True}$$

rather than creating isolated cells.

9. Region Size Constraints

The region construction process can also enforce region-size bounds:

$$2 \leq |\mathcal{R}_k| \leq 2N - 1$$

This prevents the synthesis process from creating extremely small or excessively large regions.

Region construction therefore becomes a constrained search problem rather than a simple random flood-fill.

10. Module 3 — CSP Solver

After constructing a candidate board, the system must verify whether it is actually a valid puzzle.

Let:

$$X_{r,c} \in \{0, 1\}$$

where:

$$X_{r,c} = 1$$

means that a queen is placed at cell (r, c) .

The solver enforces four constraint groups.

10.1 Row Constraint

$$\sum_{c=0}^{N-1} X_{r,c} = 1 \quad \forall r \in [0, N - 1]$$

for every row.

10.2 Column Constraint

$$\sum_{r=0}^{N-1} X_{r,c} = 1 \quad \forall c \in [0, N - 1]$$

for every column.

10.3 Region Constraint

$$\sum_{(r,c) \in \mathcal{R}_k} X_{r,c} = 1 \quad \forall k \in [0, N-1]$$

for every region.

10.4 Chebyshev Constraint

For cells at Chebyshev distance one:

$$X_{r_1, c_1} + X_{r_2, c_2} \leq 1 \quad \text{where } \max(|r_1 - r_2|, |c_1 - c_2|) = 1$$

These constraints correspond directly to the puzzle's rules.

11. Backtracking and Constraint Propagation

The solver uses depth-first search.

At each level:

1. Select the current row.
2. Try a candidate column.
3. Check column conflicts.
4. Check region conflicts.
5. Check queen adjacency.
6. Continue recursively.

Conceptually:

```
solve(row):  
    for each candidate column:  
        if valid:  
            place queen  
            solve(row + 1)  
            remove queen
```

However, the system does not need to enumerate every possible solution.

The objective is only to determine whether:

$$S = 1$$

Therefore, once:

$$S \geq 2$$

the solver immediately terminates.

The candidate is then classified as:

```
rejectedMultiSolution
```

This early termination reduces unnecessary computation.

12. Module 4 — Dihedral Symmetry Canonicalization

Two boards can be mathematically different as arrays but structurally identical.

The system therefore considers the eight transformations of the square:

$$D_4 = \{R_0, R_{90}, R_{180}, R_{270}, F_H, F_V, F_{D1}, F_{D2}\}$$

For each transformation, the system:

1. Transforms the grid.
2. Normalizes region identifiers.
3. Serializes the resulting grid.
4. Calculates the canonical minimum representation.

The canonical key is:

$$K(G) = \min_{T \in D_4} \text{Serialize}(\text{NormalizeLabels}(T(G)))$$

The key is then checked against an in-memory hash set.

If:

$$K(G) \in \text{SeenGrids}$$

the board is rejected as a structural duplicate.

Otherwise, it is added to the catalogue.

13. Asynchronous Generation

A major implementation problem is computational blocking.

Backtracking can potentially execute thousands or millions of operations.

If all of this computation occurs synchronously on the browser's main execution thread, the frontend can become unresponsive.

The project therefore uses a **cooperative asynchronous event-loop architecture**.

The generator processes a limited number of operations per execution chunk:

```
Generation Chunk
  ↓
Process K states
  ↓
Yield control
  ↓
Update UI
  ↓
Process next K states
```

This architecture allows the interface to continue responding while generation proceeds.

The design also supports long-running generation, pause/resume behavior, and state serialization.

14. Telemetry System

The generator exposes runtime statistics.

Important metrics include:

14.1 Explored

Number of partition states examined.

14.2 Accepted

Number of boards that passed uniqueness verification.

14.3 Rejected Multi-Solution

Boards for which:

$$S \geq 2$$

14.4 Rejected Duplicate

Boards that were structurally equivalent to an existing board under D_4 .

14.5 Status

The generator can report states such as:

Active
Target Reached
Exhausted
Pending

These metrics provide visibility into the behavior of the generation algorithm.

15. Frontend

The frontend acts as the presentation and interaction layer of the system.

The frontend is already implemented and provides the user-facing environment for interacting with the puzzle generation engine.

Its responsibilities include:

- Displaying the puzzle grid.
- Displaying region boundaries.
- Displaying generated queen configurations.
- Presenting generation status.
- Presenting telemetry.
- Controlling generation.
- Presenting accepted puzzles.
- Interacting with stored puzzle data.

The frontend communicates with the application's backend/data layer rather than implementing the core mathematical algorithms itself.

16. Cloud Database Integration

The implemented application is connected to a cloud database for persistent storage.

The database layer can store information such as:

```
Puzzle ID
Grid Size
Region Configuration
Queen Solution
Canonical Key
Generation Metadata
Creation Information
Verification Status
```

A conceptual data flow is:

```
Puzzle Generator
    ↓
Uniqueness Verification
    ↓
D4 Canonicalization
    ↓
Accepted Puzzle
    ↓
Cloud Database
    ↓
Frontend Retrieval
    ↓
Puzzle Display
```

This separates **puzzle synthesis** from **puzzle persistence**.

The cloud database therefore allows generated puzzles and relevant metadata to remain available beyond a single browser session.

17. Advantages of the Proposed System

Guaranteed Initial Validity

Because generation begins from a valid queen configuration, every candidate has at least one known solution before region synthesis.

Contiguous Regions

Multi-source BFS maintains orthogonal connectivity during region construction.

Exact Uniqueness Verification

The CSP solver rejects boards with multiple solutions rather than assuming that the first discovered solution is sufficient.

Structural Deduplication

D_4 canonicalization prevents rotational and reflective duplicates.

Non-Blocking Execution

Cooperative asynchronous execution prevents long-running searches from freezing the user interface.

Persistent Storage

Cloud database integration allows generated puzzles and associated metadata to persist beyond an individual browser session.

18. Challenges and Solutions

Challenge	Solution
Random partitions frequently unsolvable	Generate valid queen seeds first
Multiple solutions	Exact CSP verification
Disconnected regions	Multi-source BFS
Rotated/reflected duplicates	D_4 canonicalization
Large search trees	Constraint propagation and early termination
Browser freezing	Cooperative asynchronous execution
Runtime visibility	Generation telemetry
Persistence	Cloud database integration

19. Computational Complexity

The overall system does not have a simple polynomial runtime guarantee because the underlying problems involve combinatorial search.

19.1 N-Queens Generation

The naive permutation space is:

$$O(N!)$$

although constraint checking prunes invalid permutations considerably.

19.2 Region Synthesis

Region construction involves a constrained search over possible cell assignments. The theoretical search space can grow exponentially with the number of cells.

19.3 CSP Verification

Backtracking has exponential worst-case behavior.

For a naive row-based assignment, the search space can approach:

$$O(N!)$$

before constraint pruning.

19.4 Symmetry Processing

For each candidate, eight transformations need to be examined:

$$O(8N^2) = O(N^2)$$

with respect to board dimensions.

The system's primary performance strategy is therefore **search-space reduction**, rather than attempting to make the underlying combinatorial problem polynomial.

20. Testing and Verification

Testing should occur at multiple levels.

20.1 Unit Testing

Individual algorithms should be tested independently:

- N-Queens generator
- BFS region generator
- Region connectivity checker
- CSP solver
- Symmetry transformation
- Canonicalization function

20.2 Integration Testing

The complete pipeline should be tested:

Seed

- Region Generation
- CSP Verification
- Symmetry Filtering
- Database
- Frontend

20.3 Correctness Testing

Every accepted puzzle should satisfy:

$$S = 1$$

and:

$$\text{Connected}(\mathcal{R}_k) = \text{True}$$

for every region $\mathcal{R}_k$.

The generated queen solution must also satisfy:

- Exactly one queen per row.
 - Exactly one queen per column.
 - Exactly one queen per region.
 - $d_\infty(q_i, q_j) > 1$ for every pair of queens $i \neq j$.
-

21. Performance Evaluation

The system should be evaluated using the telemetry exposed by the engine.

Useful metrics include:

$$\text{AcceptanceRate} = \frac{\text{Accepted}}{\text{Explored}}$$

$$\text{MultiSolutionRate} = \frac{\text{RejectedMultiSolution}}{\text{Explored}}$$

$$\text{DuplicateRate} = \frac{\text{RejectedDuplicate}}{\text{Explored}}$$

and:

$$\text{GenerationThroughput} = \frac{\text{AcceptedBoards}}{\text{Time}}$$

These metrics can be recorded for different values of N , allowing the scalability of the generation engine to be evaluated.

Important: actual experimental values should be obtained from the running implementation. Proposed or target performance values should not be presented as achieved results unless experimentally verified.

22. Future Scope

22.1 Graph Neural Network Search Pruning

The grid can be represented as a graph where:

- Each cell is a vertex.
- Orthogonally adjacent cells form edges.

A GCN or MPNN could predict which region boundaries are likely to produce unique solutions.

This could reduce unnecessary search.

22.2 Automated Difficulty Estimation

Puzzle difficulty could be estimated from solving behavior.

Potential metrics include:

- Deterministic forced-move chains.
- Branching depth.
- Number of deductions.
- Backtracking depth.
- Human solving time.

These measures could later be combined with Item Response Theory for cognitive difficulty modeling.

22.3 WebAssembly Acceleration

The computationally intensive solver could be implemented in Rust or C++ and compiled to WebAssembly.

Possible optimizations include:

- Web Workers.
- SIMD.
- Bitboard representations.
- Parallel candidate evaluation.

22.4 Generative Models

A future version could employ conditional discrete diffusion models to generate complex region geometries conditioned on queen positions while incorporating topological constraints.

23. Conclusion

The **Queens Puzzle Synthesis and Verification Engine** approaches procedural logic-puzzle generation as a constrained synthesis problem rather than a random generation problem.

Its key architectural decision is to **generate the solution structure before generating the puzzle structure**. Valid N -Queens configurations provide guaranteed solution seeds, BFS wavefront expansion constructs connected regions around those seeds, exact CSP backtracking verifies uniqueness, and D_4 canonicalization eliminates geometrically equivalent puzzles.

The resulting architecture combines:

N -Queens + BFS + CSP + Backtracking + D_4 + Async Processing + Cloud Pers
--

into a single automated generation pipeline.

The technical foundation is broader than a conventional puzzle application. It demonstrates practical application of **constraint satisfaction, combinatorial search, graph traversal, computational geometry, symmetry reduction, asynchronous computation, and persistent web application architecture**.
